

Universidade do Minho
Escola de Engenharia
Licenciatura em Engenharia Informática

Unidade Curricular de Processamento de Linguagens

Ano Letivo de 2025/2026

ForTradUM

José Fernandes
a106937

Gabriel Dantas
a107291

Simão Oliveira
a107322

Maio, 2026

PL

Índice

1. Introdução	2
2. Arquitetura do Compilador	3
3. Pré-Processamento	4
4. Análise Léxica	5
4.1. Tokens Reconhecidos	5
4.2. Decisões de Implementação	5
5. Análise Sintática	7
5.1. Gramática	7
5.2. Nó CallOrArr	7
5.3. Ciclos DO	8
6. Análise Semântica	9
6.1. Tabela de Símbolos	9
6.2. Verificações Efetuadas	9
6.3. Coerção implícita INTEGER → REAL	9
6.4. Reestruturação dos ciclos DO	10
7. Representação Intermédia	11
7.1. Estrutura	11
7.2. Temporários	11
7.3. Geração de IR	12
7.3.1. Saltos Condicionais	12
7.3.2. Ciclos DO	12
7.3.3. Chamada a funções e subrotinas	13
7.3.4. Arrays	13
7.3.5. Menos Unário	14
7.3.6. Coerção implícita INTEGER → REAL	14
8. Otimização de Código	15
9. Geração de Código	16
9.1. Frame de execução	16
9.2. Reserva de espaço: PUSHN	16
9.3. Arrays	16
9.4. Operador de desigualdade NEQ	17
9.5. Funções intrínsecas	17
10. Testes	18
10.1. Estrutura	18
10.2. Testes	18
11. Dificuldades e Conclusão	20
Instruções de Execução	21
Bibliografia	22
Lista de Siglas e Acrónimos	23

1. Introdução

O presente relatório descreve o desenvolvimento do projeto proposto pela equipa docente no âmbito da unidade curricular de Processamento de Linguagens. O objetivo é construir um compilador para a linguagem Fortran 77 (standard ANSI X3.9-1978), implementado em Python com recurso às ferramentas `ply.lex` e `ply.yacc`.

O compilador recebe ficheiros Fortran 77 em formato fixo de colunas, conforme definido pelo standard [1], e produz código para a máquina virtual fornecida. Para tal, o processo de compilação foi dividido em seis etapas: análise léxica, análise sintática, análise semântica, geração de representação intermédia, otimização de código e geração de código final.

2. Arquitetura do Compilador

O compilador está organizado como um pipeline de seis etapas sequenciais, cada uma implementada num módulo Python independente. A figura seguinte ilustra o fluxo de dados entre etapas:

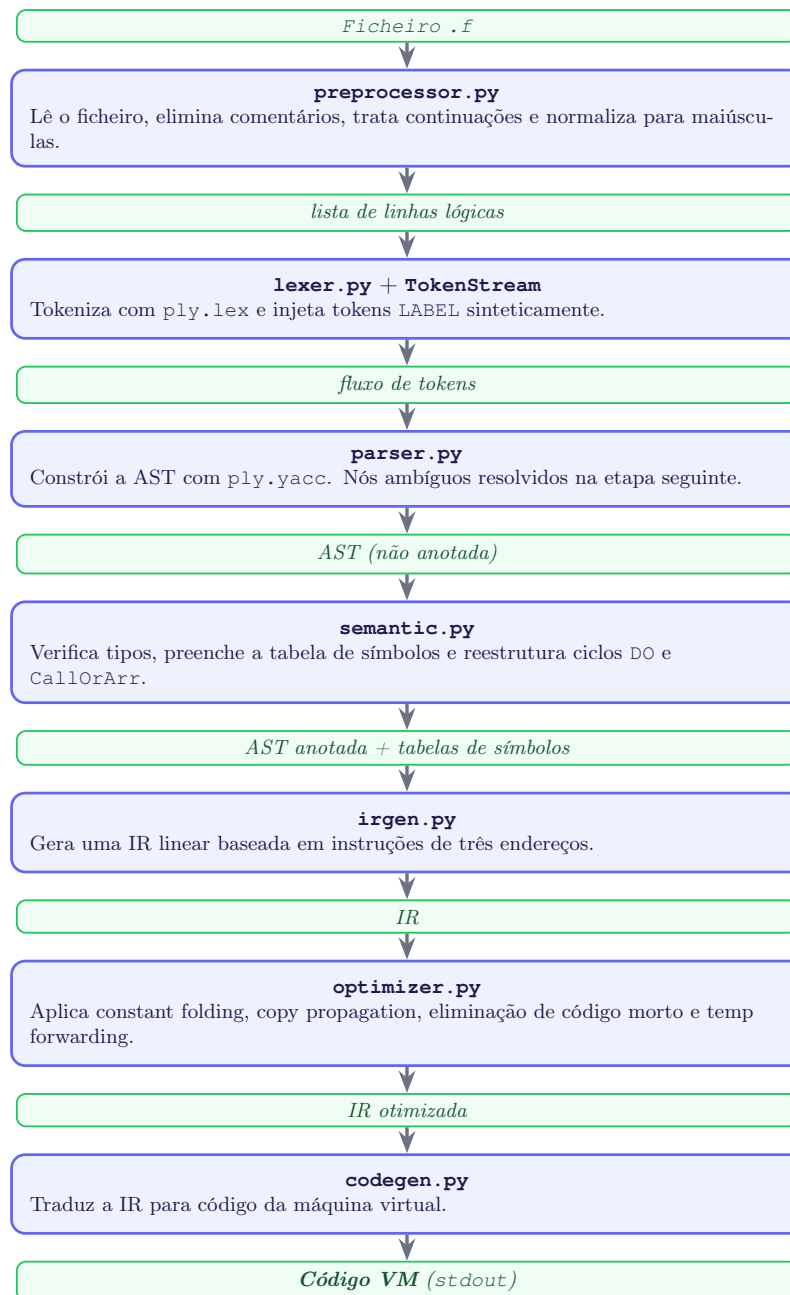


Figura 1: Pipeline do compilador

O módulo `main.py` orquestra todas as etapas, aceitando a flag `-q` para suprimir saída de depuração e imprimir apenas o código VM final.

3. Pré-Processamento

O Fortran 77 define dois formatos de ficheiro [1]: o formato fixo de colunas (original do standard ANSI X3.9-1978) e o formato livre, introduzido em versões posteriores da linguagem. O grupo optou pelo formato fixo, por ser o formato canónico do Fortran 77.

No formato fixo, cada linha física tem um significado estrutural baseado na posição das colunas:

Coluna 1	Se contiver C, c ou *, a linha é um comentário e é ignorada na totalidade
Colunas 1-5	Label de instrução — número inteiro que identifica a linha para efeitos de GOTO e ciclos DO
Coluna 6	Indicador de continuação — qualquer carácter diferente de espaço ou 0 indica que esta linha é a continuação da linha lógica anterior
Colunas 7-72	Código fonte da instrução
Colunas 73-80	Ignorado (reservado historicamente para numeração de cartões perfurados)

Tabela 1: Estrutura de colunas do formato fixo do Fortran 77

O módulo `preprocessor.py` implementa estas regras e produz uma lista de linhas lógicas, onde cada linha de continuação já foi concatenada com a sua predecessora. Cada linha lógica é representada como um dicionário com três campos: o label (se presente), o código da instrução e o número da linha física de origem (para mensagens de erro).

```
{ 'label': '10', 'code': 'CONTINUE', 'physical_line': 42 }  
{ 'label': None, 'code': 'PRINT *, X', 'physical_line': 7 }
```

Listagem 1: Exemplos de linhas lógicas produzidas pelo pré-processador

Uma decisão importante foi a de normalizar o código para maiúsculas nesta fase, antes de chegar à fase de análise léxica. O Fortran 77 é *case-insensitive*, mas tratar essa insensibilidade no léxico obrigaria a duplicar padrões. Ao converter tudo para maiúsculas no pré-processador (preservando o conteúdo de literais de cadeia de caracteres, que são delimitados por plicas e cujo conteúdo deve ser mantido tal como escrito) a análise léxica pode assumir que toda a entrada já está em maiúsculas, simplificando significativamente as expressões regulares.

4. Análise Léxica

O analisador léxico foi implementado com `ply.lex` [2] no módulo `lexer.py`. A sua função é converter cada linha lógica produzida pelo pré-processador numa sequência de *tokens*, que serão posteriormente consumidos pelo *parser*.

4.1. Tokens Reconhecidos

Os *tokens* estão divididos nas seguintes categorias:

Palavras-chave	PROGRAM, END, INTEGER, REAL, LOGICAL, CHARACTER, IF, THEN, ELSE, ENDIF, DO, CONTINUE, GOTO, READ, PRINT, RETURN, FUNCTION, SUBROUTINE, CALL
Operadores relacionais	.EQ., .NE., .LT., .LE., .GT., .GE.
Operadores lógicos	.AND., .OR., .NOT.
Literais lógicos	.TRUE., .FALSE.
Literais de valor	INTEGER_LIT, REAL_LIT, STRING_LIT
Operador de potência	POWER (**)
Caracteres simples	(,), ,, =, +, -, /, *
Identificadores	ID

Tabela 2: Categorias de tokens reconhecidos pelo analisador léxico

4.2. Decisões de Implementação

- Palavras-chave vs. identificadores** – As palavras-chave são reconhecidas dentro da regra `t_ID`: o *lexer* tenta fazer corresponder qualquer sequência de letras e dígitos a um identificador, e só depois consulta um dicionário de palavras-chave para determinar o tipo correto do token. Esta abordagem é a recomendada pelo PLY e evita conflitos entre padrões.
- Operador de potência** – O token `POWER (**)` é definido como função em vez de string, o que garante que o PLY lhe atribui maior prioridade do que ao literal `*`. Sem esta precaução, `**` seria *tokenizado* como dois `*` separados.
- Literais reais antes de inteiros** – A regra `t_REAL_LIT` é definida antes de `t_INTEGER_LIT`. O PLY ordena as regras definidas como strings por comprimento decrescente, mas as regras definidas como funções são testadas pela ordem em que aparecem no ficheiro. Assim, a sequência `3.14` é corretamente reconhecida como `REAL_LIT` e não como o inteiro `3` seguido de `.14`.
- Operadores ponto** – Os operadores relacionais e lógicos do Fortran são delimitados por pontos (`.EQ.`, `.AND.`, etc.) e definidos como strings simples. Como o PLY ordena as regras por comprimento decrescente, não há risco de ambiguidade entre eles.
- Token Label** – O token `LABEL` não é produzido pelo *lexer* diretamente. Os labels de instrução estão nas colunas 1–5 da linha física, que o pré-processador já separou do corpo da instrução. O módulo `main.py` define uma classe `TokenStream` que, para cada linha lógica com label, injeta sinteticamente um token

LABEL antes dos restantes tokens dessa linha. O parser recebe assim um fluxo unificado sem precisar de conhecer a estrutura de colunas do formato fixo.

5. Análise Sintática

O analisador sintático foi implementado com `ply.yacc` [2] no módulo `parser.py`. A sua função é consumir o fluxo de tokens produzido pela *TokenStream* e construir a AST, composta pelos nós definidos em `ast_nodes.py`.

5.1. Gramática

A gramática implementada cobre as seguintes construções do Fortran 77:

1. **Unidades de programa** – Um ficheiro pode conter um programa principal seguido de zero ou mais subprogramas (funções e subrotinas). O programa principal pode ou não ter a instrução `PROGRAM nome`.
2. **Declarações de tipo** – As declarações `INTEGER`, `REAL`, `LOGICAL` e `CHARACTER` aceitam listas de variáveis, onde cada variável pode ser escalar ou array. O tamanho do array pode ser um literal inteiro (array estático) ou um identificador (array dinâmico, tipicamente um parâmetro da subrotina).
3. **Instruções de controlo de fluxo** – São suportadas as construções `IF-THEN-ELSE-ENDIF`, `GOTO` e ciclos `DO` com label terminal. O cabeçalho do ciclo `DO` aceita opcionalmente um passo (step), que assume o valor 1 por omissão.
4. **Expressões** – As expressões aritméticas, relacionais e lógicas são tratadas com a tabela de precedências do `PLY`, que resolve ambiguidades evitando a necessidade de múltiplas regras não-ambíguas para cada nível de prioridade de operador. A tabela de precedências, da menor para a maior, é a seguinte:

Nível	Associatividade	Operadores
1 (menor)	esquerda	.OR.
2	esquerda	.AND.
3	direita	.NOT.
4	não-assoc	.EQ. .NE. .LT. .LE. .GT. .GE.
5	esquerda	+ -
6	esquerda	* /
7	direita	UMINUS (menos unário)
8 (maior)	direita	POWER (**)

Tabela 3: Tabela de precedências do parser

5.2. Nó CallOrArr

Uma ambiguidade inerente ao Fortran 77 é que a sintaxe `ID(expr)` pode representar tanto um acesso a um elemento de array como uma chamada a uma função. O parser não tem informação suficiente para distinguir os dois casos, pelo que produz um nó intermédio `CallOrArr` sempre que encontra esta construção. A resolução é feita posteriormente pelo analisador semântico, que consulta a tabela de símbolos para determinar se o identificador é um array ou uma função, substituindo o nó por `ArrRef` ou `CallExpr` conforme o caso.

5.3. Ciclos DO

O Fortran 77 em formato fixo escreve os ciclos DO de forma *flat*, ou seja, o cabeçalho DO 10 I = 1, N e o 10 CONTINUE que o termina estão ao mesmo nível na lista de instruções, sem qualquer delimitador explícito do corpo. O parser limita-se a reconhecer o cabeçalho DO como uma instrução isolada, deixando a reestruturação em árvore aninhada para o analisador semântico.

```
def p_do_stmt(p):
    ...
    do_stmt : DO INTEGER_LIT ID '=' expr ',' expr ',' expr
            | DO INTEGER_LIT ID '=' expr ',' expr
    ...
    if len(p) == 10:
        p[0] = Do(label=p[2], var=p[3], start=p[5], end=p[7], step=p[9])
    else:
        p[0] = Do(label=p[2], var=p[3], start=p[5], end=p[7], step=None)

def p_continue_stmt(p):
    'continue_stmt : CONTINUE'
    p[0] = Continue()
```

Listagem 2: Cabeçalho DO e terminador CONTINUE são instruções isoladas no parser

6. Análise Semântica

O analisador semântico foi implementado no módulo `semantic.py` e é responsável por verificar a coerência do programa, preencher a tabela de símbolos e transformar a AST produzida pelo parser numa forma anotada pronta para a geração de código.

6.1. Tabela de Símbolos

Para cada unidade de programa (programa principal, função ou subrotina), é construída uma tabela de símbolos independente. Cada entrada associa o nome da variável a um dicionário com os seguintes campos:

```
{ 'type': 'INTEGER', 'kind': 'scalar' }
{ 'type': 'REAL', 'kind': 'array', 'size': 5 }
{ 'type': 'INTEGER', 'kind': 'dynamic_array', 'size': 'N' }
{ 'type': 'INTEGER', 'kind': 'param' }
{ 'type': 'INTEGER', 'kind': 'array_param', 'size': 'N' }
```

Listagem 3: Exemplo de entradas na tabela de símbolos

O campo `kind` distingue entre variáveis escalares, arrays estáticos, arrays dinâmicos (cujo tamanho é um parâmetro), parâmetros escalares e parâmetros array.

6.2. Verificações Efetuadas

O analisador semântico realiza as seguintes verificações:

1. **Variáveis não declaradas** – qualquer referência a um identificador não presente na tabela de símbolos é reportada como erro.
2. **Declarações duplicadas** – uma variável não pode ser declarada mais do que uma vez na mesma unidade de programa.
3. **Tipos incompatíveis** – operações entre tipos incompatíveis são reportadas como erro, com exceção da mistura `INTEGER/REAL`, que é resolvida por conversão implícita.
4. **Labels indefinidos** – referências `GOTO` e cabeçalhos `DO` que apontam para labels não definidos no mesmo âmbito são reportadas como erro.
5. **Assinaturas de subprogramas** – o número de argumentos passados a funções e subrotinas é verificado contra a sua declaração. Um `CALL` a uma função (em vez de subrotina) é também reportado como erro.

6.3. Coerção implícita `INTEGER` → `REAL`

O Fortran 77 permite misturar `INTEGER` e `REAL` em expressões aritméticas, convertendo implicitamente o operando inteiro para real. O analisador semântico detecta estas situações e insere nós `Coerce` na AST para tornar a conversão explícita:

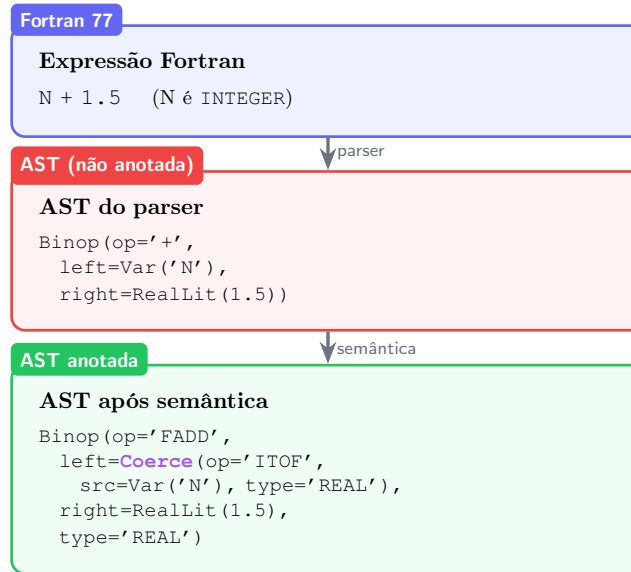


Figura 2: Inserção de nó Coerce numa expressão mista INTEGER

6.4. Reestruturação dos ciclos DO

Como descrito na secção anterior, o parser produz os ciclos DO como instruções isoladas numa lista *flat*. O analisador semântico percorre essa lista mantendo uma pilha de ciclos abertos. Quando encontra um DO, empurra um novo *frame* para a pilha. As instruções seguintes são acumuladas no corpo do *frame* mais recente. Quando encontra um Labeled(Continue()) cujo label coincide com o do frame no topo da pilha, fecha o ciclo e produz um nó DoLoop com o corpo aninhado. Ciclos DO que partilham o mesmo label terminal são todos fechados de uma vez, suportando assim a sintaxe de ciclos aninhados do Fortran 77.

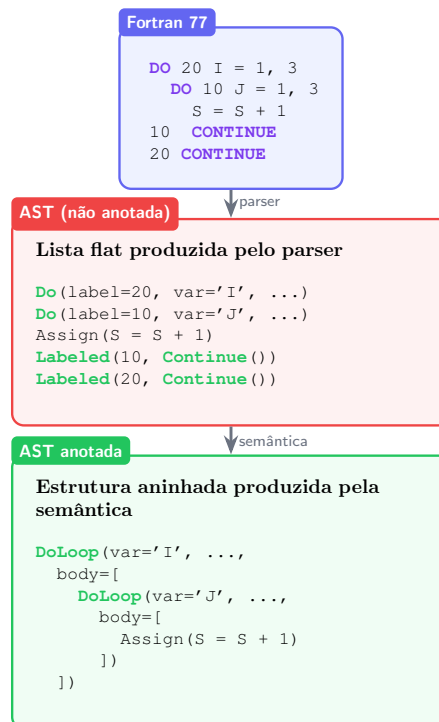


Figura 3: Reestruturação de ciclos DO pelo analisador semântico

7. Representação Intermédia

Após a análise semântica, o compilador gera uma IR linear, implementada nos módulos `ir.py` e `irgen.py`. A IR serve de camada de abstração entre a AST e o código final da máquina virtual, facilitando a aplicação de otimizações.

7.1. Estrutura

A IR é organizada em procedimentos, um por unidade de programa (programa principal, função ou subrotina). Cada procedimento contém a sua tabela de símbolos, metadados (nome, tipo, tipo de retorno) e uma lista linear de instruções.

As instruções IR são *dataclasses* Python, cada uma representando uma operação atômica:

<code>Copy(dst, src)</code>	Copia um valor para uma variável
<code>Binop(dst, op, left, right)</code>	Operação binária entre dois operandos
<code>Unary(dst, op, src)</code>	Operação unária
<code>Coerce(dst, op, src)</code>	Conversão de tipo (ex: IT0F)
<code>LoadArr(dst, name, idx)</code>	Leitura de elemento de array
<code>StoreArr(name, idx, val)</code>	Escrita em elemento de array
<code>Read(var)</code>	Leitura de valor da entrada padrão
<code>ReadArr(name, idx)</code>	Leitura de valor da entrada padrão para array
<code>Print(vals)</code>	Impressão de uma lista de valores
<code>Label(name)</code>	Marca um ponto de salto no código
<code>Jump(label)</code>	Salto incondicional
<code>Jz(label, cond)</code>	Salto condicional se cond for zero
<code>Call(dst, name, args)</code>	Chamada a função ou subrotina
<code>Return(val)</code>	Retorno de subprograma

Tabela 4: Instruções da IR

7.2. Temporários

As expressões são decompostas em operações atômicas com recurso a variáveis temporárias geradas automaticamente (`t0`, `t1`, `t2`, ...). Cada temporário é usado exatamente para armazenar o resultado de uma operação intermédia. Por exemplo:

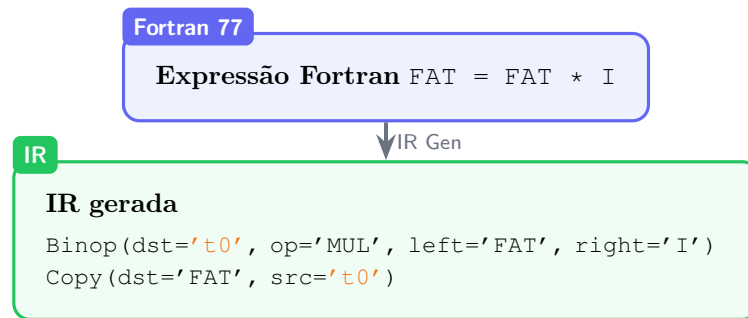


Figura 4: Decomposição de uma expressão em instruções IR com temporários

7.3. Geração de IR

O módulo `irgen.py` percorre a AST anotada recursivamente. As expressões são geradas pela função `gen_expr`, que devolve o nome do temporário (ou valor imediato) que contém o resultado. As instruções são geradas pela função `gen_stmt`, que emite instruções IR diretamente para o procedimento corrente.

A maioria das construções do Fortran 77 traduz-se de forma direta para instruções IR — atribuições, operações aritméticas e lógicas, leituras e impressões seguem um padrão simples de empilhar operandos e emitir a instrução correspondente. Nas subsecções seguintes descrevem-se apenas as construções cuja geração envolve decisões não óbvias.

7.3.1. Saltos Condicionais

Os saltos condicionais são gerados a partir de nós `If` da AST. Para um IF sem ELSE, é gerado um `Jz` para o label de fim do bloco. Para um IF com ELSE, são gerados dois labels, um para o início do bloco ELSE e outro para o fim, juntamente com um `Jump` no final do bloco THEN:

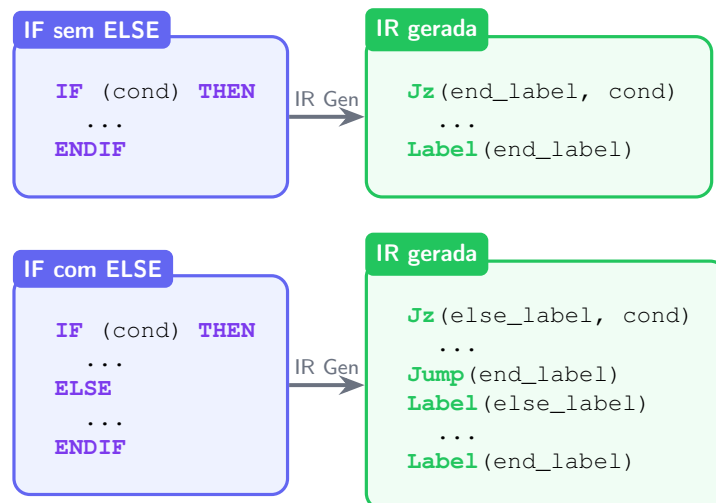


Figura 5: Geração de IR para IF-THEN-ELSE

7.3.2. Ciclos DO

A condição de saída de um ciclo DO no Fortran 77 não é uma simples comparação entre a variável de iteração e o limite final. O standard [1] define que o ciclo executa enquanto $(end - var) * step \geq 0$, o que garante o comportamento correto para passos negativos. A IR gerada calcula esta condição explicitamente a cada iteração:

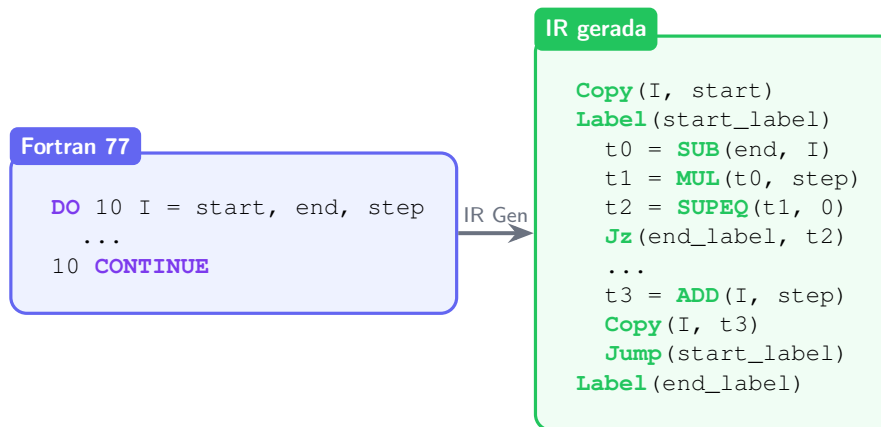


Figura 6: Geração de IR para ciclo DO

Os valores de *end* e *step* são avaliados uma única vez antes do início do ciclo e guardados em temporários, conforme o standard exige.

7.3.3. Chamada a funções e subrotinas

A convenção de chamada adotada segue o modelo da máquina virtual. O *caller* reserva um *slot* extra na *stack* antes dos argumentos com `PUSHI 0`, este *slot* serve de espaço de retorno. A função escreve o valor de retorno nesse *slot* através de `STOREL -(nParams+1)`, que corresponde ao *offset* negativo do *slot* relativamente ao *frame pointer*. No final, o *caller* remove os argumentos da *stack* com `POP n` e lê o resultado do *slot* de retorno.

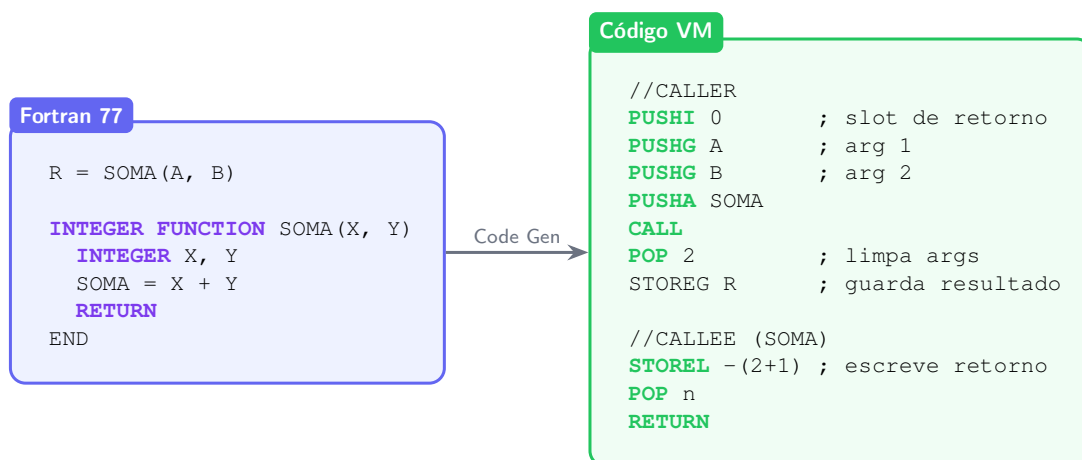


Figura 7: Convenção de chamada de funções

As subrotinas seguem o mesmo modelo mas sem *slot* de retorno — o `CALL` é emitido sem `PUSHI 0` e o `RETURN` emite apenas `POP n` seguido de `RETURN`.

7.3.4. Arrays

Os arrays estáticos são alocados no *heap* da VM com `ALLOC n`, onde *n* é o tamanho declarado. Os arrays dinâmicos, cujo tamanho é um parâmetro da subrotina, usam `ALLOCN`, que lê o tamanho da *stack*. Em ambos os casos, o endereço base do array é guardado numa variável local ou global.

O acesso a um elemento `A(I)` é feito calculando o endereço do elemento com `PADD`, após decrementar o índice em 1, pois o Fortran usa arrays indexados a partir de 1 e a VM indexa a partir de 0:

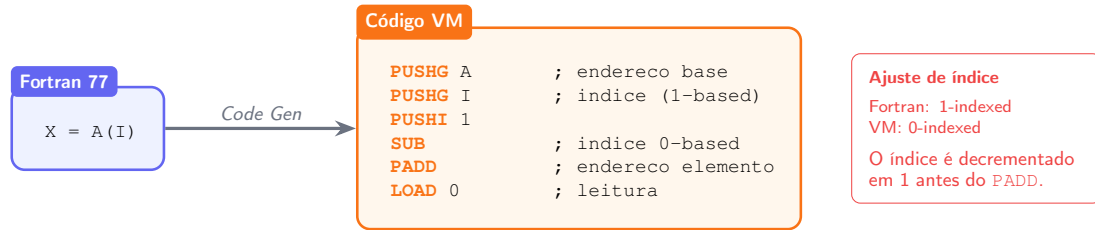


Figura 8: Acesso a elemento de array A(I)

7.3.5. Menos Unário

A máquina virtual não possui um *opcode* de negação unária. O compilador resolve isto reescrevendo $-x$ como $0 - x$, usando o *opcode* SUB para inteiros ou FSUB para reais, com zero do tipo correto como operando esquerdo:

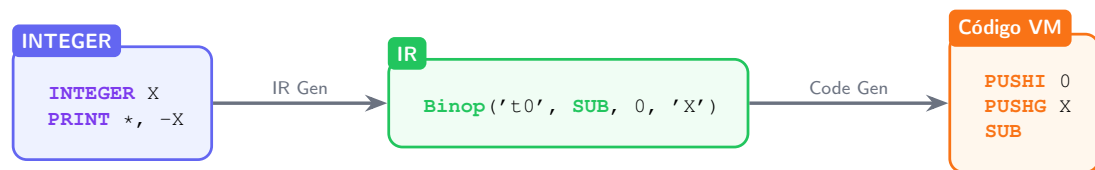


Figura 9: Reescrita do menos unário em IR

7.3.6. Coerção implícita INTEGER → REAL

Como mencionado anteriormente, quando os dois operandos de uma operação aritmética têm tipos diferentes (um INTEGER e um REAL o analisador semântico insere um nó Coerce na AST para converter o operando inteiro para real. A IR gerada emite o *opcode* ITOF da VM antes da operação:

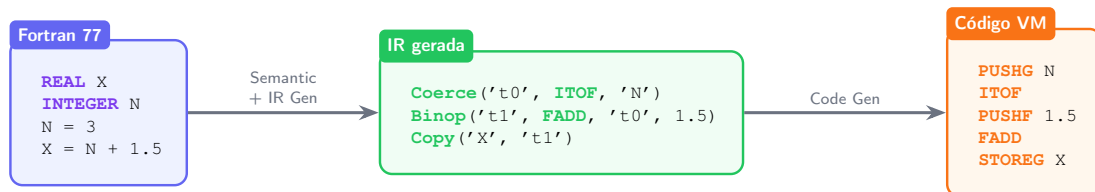


Figura 10: Geração de IR para coerção implícita

8. Otimização de Código

Após a geração da IR, o módulo `optimizer.py` aplica um conjunto de otimizações locais sobre cada procedimento. As otimizações são executadas num ciclo fixo — repetido até que nenhuma alteração seja produzida — pela seguinte ordem:

1. **Constant Folding** — expressões cujos dois operandos são valores literais conhecidos em tempo de compilação são avaliadas diretamente, substituindo a instrução `Binop` por uma instrução `Copy` com o resultado calculado. Por exemplo, `Binop('t0', MUL, 2, 3)` é substituído por `Copy('t0', 6)`.
2. **Temp Forwarding** — quando um temporário é produzido por uma instrução e consumido exactamente uma vez, na instrução imediatamente seguinte, numa `Copy`, o temporário intermédio é eliminado e a instrução produtora escreve directamente no destino final.
3. **Copy Propagation** — após uma instrução `Copy(dst, src)`, todas as utilizações subsequentes de `dst` são substituídas por `src`, enquanto `dst` não for reatribuído. Isto elimina variáveis temporárias desnecessárias e encurta cadeias de cópias.
4. **Dead Code Elimination** — temporários que são escritos mas nunca lidos são removidos juntamente com as instruções que os produzem.

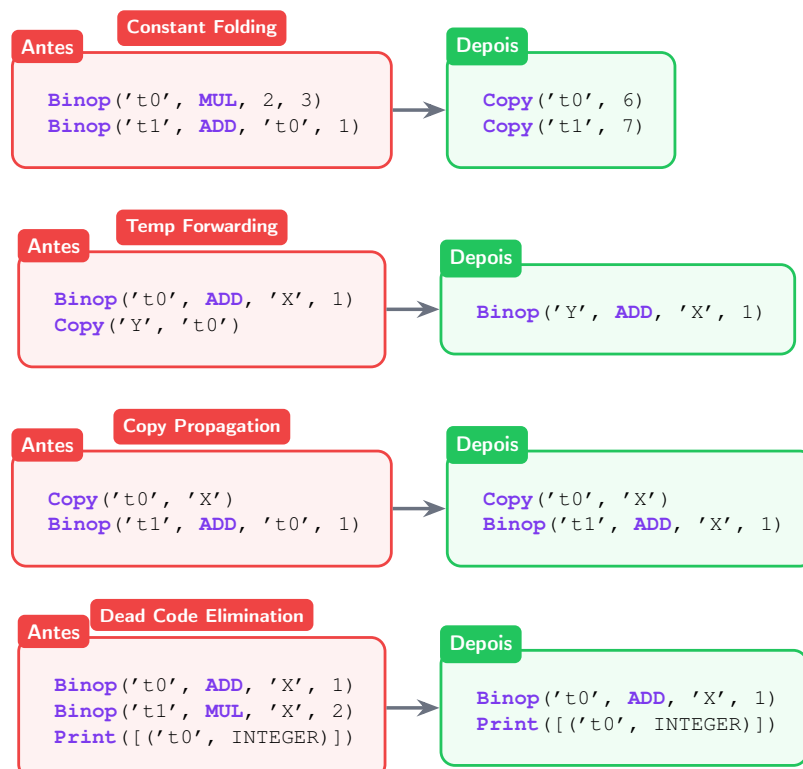


Figura 11: Exemplos das quatro otimizações aplicadas pelo compilador

A aplicação repetida destas quatro otimizações em conjunto é mais poderosa do que qualquer uma isolada, a *constant folding* pode expor oportunidades de *copy propagation*, que por sua vez pode tornar temporários mortos elegíveis para eliminação.

9. Geração de Código

O módulo `codegen.py` traduz a IR otimizada para código da máquina virtual, procedimento a procedimento, pela classe `CodeGenerator`.

9.1. Frame de execução

Para cada procedimento, o método `buildFrame` percorre a tabela de símbolos produzida pela análise semântica e atribui a cada variável um offset na frame de execução. O tratamento difere consoante o tipo de procedimento:

- No programa principal, todas as variáveis são globais e acedidas com `PUSHG / STOREG`. Os offsets são atribuídos sequencialmente a partir de 0, pela ordem em que as variáveis aparecem na tabela de símbolos.
- Em funções e subrotinas, as variáveis locais são acedidas com `PUSHL / STOREL`. Os parâmetros recebem offsets negativos relativamente ao frame pointer — com n parâmetros, o primeiro fica em `fp[-n]` e o último em `fp[-1]` — porque foram empilhados pelo caller antes do `CALL`. As variáveis locais recebem offsets positivos a partir de 0.

Esta distinção é controlada pela flag `useLocal` da classe `CodeGenerator`, que é activada para qualquer procedimento que não seja o programa principal.

9.2. Reserva de espaço: PUSHN

O `PUSHN n` reserva n slots na stack inicializados a zero. O valor de n é calculado em dois passos antes de qualquer instrução ser emitida:

1. `buildFrame` percorre a tabela de símbolos e atribui offsets a todas as variáveis declaradas, contabilizando o número de slots necessários para as variáveis locais.
2. `collectTemps` percorre todas as instruções IR do procedimento e recolhe o conjunto de temporários utilizados, atribuindo-lhes offsets a seguir às variáveis declaradas.

O valor final de n é a soma dos dois. Desta forma, um único `PUSHN` no início de cada procedimento reserva espaço tanto para variáveis declaradas como para temporários.

9.3. Arrays

Os arrays estáticos são inicializados logo após o `PUSHN`, com `ALLOC n` onde n é o tamanho declarado. O endereço base devolvido pela alocação é imediatamente guardado na variável correspondente. Os arrays dinâmicos, cujo tamanho depende de um parâmetro da subrotina, usam `ALLOCN`, que lê o tamanho do topo da stack antes de alocar:

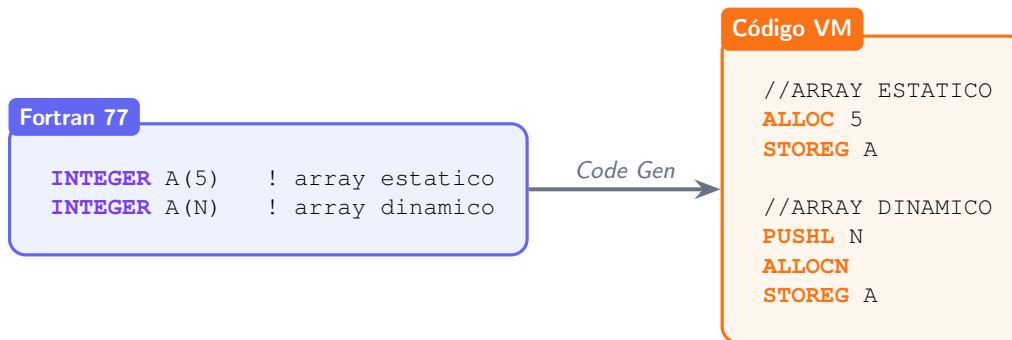


Figura 12: Inicialização de arrays estático e dinâmico

9.4. Operador de desigualdade NEQ

A máquina virtual não possui um opcode NEQ. O compilador resolve isto emitindo EQUAL seguido de NOT, o que produz o resultado correcto sem necessidade de instrução adicional na VM.

9.5. Funções intrínsecas

O Fortran 77 define um conjunto de funções matemáticas intrínsecas que o compilador suporta diretamente. Algumas mapeiam para opcodes da VM, outras têm o tipo de retorno inferido pela análise semântica mas não possuem opcode direto na VM:

MOD	MOD	Resto da divisão inteira
INT	FTOI	Conversão REAL para INTEGER por truncagem
SQRT	—	Não suportado directamente pela VM
ABS	—	Tipo de retorno igual ao do argumento
MAX	—	Tipo de retorno igual ao do argumento
MIN	—	Tipo de retorno igual ao do argumento

Tabela 5: Mapeamento de funções intrínsecas para código VM

10. Testes

10.1. Estrutura

Os testes são automatizados com `pytest` e implementados em `tests/test_compiler.py`. Cada teste compila um ficheiro `.f` com o compilador e compara a saída produzida com um ficheiro `.expected` correspondente, que contém o código VM esperado. O mecanismo é o seguinte:

```
result = subprocess.run(
    [sys.executable, str(MAIN), "-q", str(src)],
    capture_output=True, text=True
)
assert result.returncode == 0
assert result.stdout == expected_file.read_text()
```

Listagem 4: Estrutura de um teste automatizado

A flag `-q` suprime a saída de depuração do compilador, garantindo que apenas o código VM é comparado. Os testes são recolhidos automaticamente, qualquer ficheiro `.f` em `tests/` ou `tests/extra/` com um `.expected` correspondente é incluído na suite.

Para correr a suite completa usa-se:

```
pytest tests/
# ou para um ficheiro especifico
python src/main.py tests/hello.f
```

10.2. Testes

Os programas de teste cobrem as funcionalidades essenciais do compilador. O conjunto de testes foi desenvolvido em parte com recurso a ferramentas de inteligência artificial, que auxiliaram na geração de casos de teste para funcionalidades específicas e situações de fronteira.

Ficheiro	Funcionalidade testada
hello.f	Literal de cadeia, PRINT
fatorial.f	Ciclo D0, acumulador inteiro, READ
primo.f	LOGICAL, GOTO, MOD, IF-THEN-ELSE aninhado
somaarr.f	Array estático, READ para array, acumulador em ciclo D0
conversor.f	INTEGER FUNCTION, chamada em expressão, GOTO como ciclo
funcall.f	Chamada a INTEGER FUNCTION com dois argumentos
dynarr.f	Array dinâmico passado como parâmetro de subrotina
subroutine.f	SUBROUTINE com CALL e passagem de argumento
coerce.f	Coerção implícita INTEGER → REAL em expressão mista
do_step.f	Ciclo D0 com passo diferente de 1
nested_do.f	Ciclos D0 aninhados com labels distintos

logical_ops.f	Operadores .OR. e .NOT., literal .FALSE.
ge_ne.f	Operadores relacionais .GE., .NE. e .LT.
unary_minus.f	Menos unário aplicado a variável e a expressão
real_arith.f	Aritmética em vírgula flutuante (REAL)

Tabela 6: Conjunto de testes

11. Dificuldades e Conclusão

Uma das dificuldades mais significativas foi a compreensão do funcionamento da máquina virtual. A documentação disponível nem sempre era suficientemente clara quanto ao comportamento de certas instruções, em particular no que diz respeito à gestão da stack em chamadas a subprogramas. Foi necessário experimentar e validar o comportamento da VM empiricamente para perceber detalhes como o layout da frame de execução e a forma correta de implementar o slot de retorno de funções.

A construção da gramática exigiu também bastante trabalho. O Fortran 77 tem várias construções sintáticas que não se encaixam naturalmente numa gramática LALR(1) sem cuidado, a ausência de delimitadores explícitos para o corpo dos ciclos DO, a ambiguidade entre chamadas a funções e acessos a arrays, e a necessidade de introduzir o token LABEL sinteticamente para lidar com os labels de instrução nas colunas 1–5 foram pontos que obrigaram a várias iterações até se encontrar uma gramática limpa e funcional.

A construção `ID(expr)` foi um caso particularmente interessante. Embora a solução adotada (o nó intermédio `CallOrArr` resolvido na análise semântica por consulta à tabela de símbolos) não tenha sido difícil de implementar, obrigou a pensar cuidadosamente sobre a fronteira de responsabilidades entre o parser e a semântica.

A reestruturação dos ciclos DO foi outro ponto delicado. O formato flat do Fortran 77, onde o cabeçalho DO e o CONTINUE terminal estão ao mesmo nível na lista de instruções, obrigou a implementar um mecanismo de pilha na análise semântica para reconstruir corretamente a estrutura aninhada, incluindo o caso de ciclos que partilham o mesmo label terminal.

Não obstante estas dificuldades, o projeto permitiu desenvolver um compilador completo para a linguagem Fortran 77, capaz de processar os programas de exemplo do enunciado e um conjunto alargado de casos de teste adicionais. Todas as etapas obrigatórias foram implementadas: análise léxica, análise sintática, análise semântica, geração de uma IR e geração de código. Foram ainda desenvolvidas etapas de valorização, nomeadamente o suporte a funções e subrotinas e a otimização de código.

O desenvolvimento do projeto consolidou conhecimentos sobre a construção de compiladores e sobre o funcionamento de uma máquina virtual baseada em stack, e permitiu aplicar de forma prática os conceitos de processamento de linguagens estudados na unidade curricular. A organização do compilador em módulos independentes com interfaces bem definidas revelou-se uma decisão acertada, facilitando a depuração e a extensão do sistema ao longo do desenvolvimento.

Como trabalho futuro, seria interessante estender o suporte a mais construções do Fortran 77, como o tipo CHARACTER com operações de cadeia, instruções FORMAT e WRITE, e a passagem de arrays por referência de forma mais geral. A implementação de otimizações globais, como a eliminação de subexpressões comuns ou a alocação de registos, seria também uma extensão natural ao trabalho realizado.

Instruções de Execução

O compilador requer Python 3 e a biblioteca PLY, instalável com:

```
pip install ply
```

Para compilar um ficheiro Fortran 77:

```
python src/main.py <ficheiro.f>
```

Para correr a suite de testes:

```
pytest tests/
```

Bibliografia

- [1] «American National Standard Programming Language FORTRAN», n.º ANSI X3.9-1978. 1978.
- [2] D. M. Beazley, «PLY (Python Lex-Yacc)». 2011.

Lista de Siglas e Acrónimos

AST *Abstract Syntax Tree* — Árvore Sintática Abstrata

ANSI *American National Standards Institute*

IR *Intermediate Representation* — Representação Intermédia

LALR *Look-Ahead Left-to-Right Rightmost Derivation* — classe de gramáticas usada pelo PLY

PLY *Python Lex-Yacc* — biblioteca Python para construção de analisadores léxicos e sintáticos

VM *Virtual Machine* — Máquina Virtual